



DSA Tutorial

Last Updated : 20 May, 2026

DSA stands for **D**ata **S**tructures and **A**lgorithms. Data structures manage how data is stored and accessed. Algorithms focus on processing this data. Examples of data structures are Array, Linked List, Tree and Heap, and examples of algorithms are Binary Search, Quick Sort and Merge Sort.

- Foundation for almost every software like GPS, Search Engines, AI ChatBots, Gaming Apps, Databases, Web Applications, etc
- Top Companies like Google, Microsoft, Amazon, Apple, Meta and many other heavily focus on DSA in interviews.
- Learning DSA boosts your problem-solving abilities and make you a stronger programmer.

Step by Step Learning

It is advised to skip the hard problems of every section in the first iteration if you are a complete beginner.

Fundamentals

- Programming : [Input and Output](#), [Conditional Statements](#), [For loop](#), [While loop](#), [Function](#), [Classes and Objects](#)
- Complexity Analysis : [Order of Growth](#), [Asymptotic Analysis](#), [Big-O](#), [Theta](#), [Big – \$\Omega\$](#) , [Time Complexity](#), [Space Complexity](#)

Maths, Pattern & Recursion

- Theory : [Recursion](#), [Analysis of Recursion](#)
- Easy Maths : [Even or Odd](#), [Sum of Naturals](#), [Closest Number](#), [Sum of Consecutive](#).
- Easy Pattern : [Solid Rectangle](#), [Floyd's Triangle](#), [Hollow rectangle](#)
- Easy Recursion: [Print 1 to n](#), [Print n to 1](#), [Factorial](#), [GCD](#), [Power](#)
- Medium Pattern : [Pyramid](#), [Hollow Diamond](#)
- Medium Maths: [Count Digits](#), [Prime Testing](#), [Armstrong Number](#), [Trailing Zeros](#), [Prime Factors](#), [All Divisors](#)
- Hard : [Butterfly](#), [Palindrome Number](#), [Tower of Hanoi](#), [Non-Attacking Knights](#), [Josephus](#), [Water Jug](#)

Array & String

- Theory : [Array](#), [String](#), [Matrix](#)
- Easy Array : [Is Sorted](#), [Multiply with Adjacent](#), [Reverse](#), [Reverse in Groups](#), [Rotate](#), [Generate All Subarrays](#), [Arrange by Sign](#), [Stock with 1 Transaction](#), [Stock with Multiple Transactions](#), [Leaders](#)
- Easy String : [Same Strings](#), [Palindrome](#), [Toggle Case](#), [Remove Occurrences](#), [Remove Spaces](#), [Check Subsequence](#), [First Non-Repeating](#), [Pangram](#)
- Medium Array : [Majority Element](#), [Majority Element II](#), [Sorted subsequence of 3](#), [Count Strictly Increasing Subarrays](#), [Next Permutation](#), [Kadane's Algorithm](#), [Sum of all subarrays](#), [Max Product Subarray](#)
- Medium String: [Implement atoi](#), [URLify](#), [Multiply Large Numbers](#)
- 2D Array : [Diagonal Traversal](#), [Magic Square](#), [Boundary Traversal](#), [Matrix Spiral](#), [Toeplitz Matrix](#), [Matrix Zig-Zag](#), [Transpose of Matrix](#), [Rotate a matrix by 90](#), [Conway's Game Of Life](#), [Queries in a Matrix](#), [Set Matrix 0](#)
- Hard : [Max Circular Subarray Sum](#), [Next Smallest Palindrome](#), [Lexicographic Rank](#), [Text Justification](#)

Searching

- Theory : [Searching](#)
- Linear Search : [Largest](#), [Second Largest](#), [Local Min & Max](#)
- Binary Search : [Insertion Position](#), [Lower Bound](#), [Upper Bound](#), [Single Element](#), [Missing Natural](#), [Count Occurrences](#), [2D Search](#), [Sorted & Rotated](#), [Sorted & Rotated II](#), [K Closest](#), [Search in Almost Sorted](#), [Peak](#), [Peak in 2D](#), [Range based Counts](#), [Kth Missing](#), [Search in Row-Column Sorted](#)
- Search on answer: [Square Root](#), [Nth Root](#), [Check Power](#), [Pick K](#), [N trailing zeroes](#), [Book Allocation](#), [Koko Eating Banana](#), [Aggressive Cows](#), [Median in a Row Sorted](#), [Kth Smallest in Row and Column Sorted](#), [M bouquets](#), [kth in Multiplication Table](#), [Maximize Median](#), [Min Time for Orders](#), [Equalize Towers](#)
- Search on Two: [Median of two sorted](#), [Kth of two sorted](#)

Sorting

- Theory : [Sorting](#)
- Easy : [Wave Form](#), [Sort by Length](#), [Min Diff Pair](#)
- Medium : [Merge Overlapping](#), [Form Largest](#), [Tywin's War Strategy](#), [Candy Cost](#), [Meeting Rooms](#), [Insert Merge Intervals](#), [Minimize Moves](#), [Max sum of \$i \cdot arr\[i\]\$](#)
- Partition & Quick Sort Based : [Quick Sort](#), [Sort Binary](#), [Zeroes to End](#), [Sort Ternary](#)
- Merge & Merge Sort Based : [Merge Sort](#), [Union of 2 Sorted](#), [Intersection of 2 Sorted](#), [Merge Without Extra Space](#), [Minimum Platforms](#), [Inversion count](#), [Surpasser Count](#)
- Cycle Sort Based : [Cycle Sort](#), [Minimum Missing Positive](#), [Min Swaps to Sort](#)

Bit Manipulation

- Theory : [Bitwise](#)

- Easy : [Swap without Third](#), [Kth Set Bit](#), [Only set bit](#), [Power of two](#), [Sort by Set bits](#)
- Medium : [Power of 4](#), [Binary is palindrome](#), [Rotate Bits](#), [Swap Odd Even](#), [Count Set](#), [Count Total Set](#), [Iterative Power](#)
- Bit with Arrays: [Max Consecutive 1s](#), [Missing](#), [One Odd](#), [Two Odds](#), [Palindrome Permutation](#), [Missing & Repeating](#), [Unique Numbers II](#), [n-bit Gray Codes](#), [Hamming Distance](#), [Sum of Pair XORs](#), [Sum of Subset XORs](#), [Subarray XORs](#), [Is Valid Suduko](#)
- Subsets : [Subsets](#), [Subsequences](#)

Hashing

- Theory : [Hashing](#)
- Basic Hashing: [Linear Probing](#), [Separate Chaining](#), [Quadratic Probing](#), [Distinct in Array](#), [Is Subset](#), [Union](#), [Intersection](#), [2 Sum](#), [Fizz Buzz](#), [Pair Sums Divisible by k](#), [Roman to Integer](#), [Isomorphic Strings](#)
- Hashing for Frequency: [Count Frequencies](#), [Most Frequent](#), [Max Subsets](#), [Anagrams](#)
- Subsequence : [Longest Consecutive Subsequence](#), [Consecutive Subsequence](#)

Two-Pointer

- Theory : [Two-Pointer](#)
- Easy : [2 Sum in Sorted](#), [Smallest Subarray](#), [Unique Elements](#), [Sentence Palindrome](#)
- Medium : [3 Sum](#), [Celebrity Problem](#), [Closest Pair from 2](#), [Container with Most Water](#), [Common in 3](#), [Reverse Words](#), [Policemen catch thieves](#)
- Hard : [4 Sum](#), [Closest Triplet](#), [Count Triplets](#)

Sliding Window

- Theory : [Sliding Window](#)
- Fixes Length Window: [Max Sum Subarray of size K](#), [Max XOR of K size](#), [Distinct In Every Window](#), [MEX Of Subarrays](#), [Min Swaps to group all 1's](#), [First negative in every window](#), [Chocolate Distribution](#)
- Variable Length : [Maximize Ones](#), [Equal substring with cost less than K](#), [Fruit Into Baskets](#), [Subarray with given sum](#)
- Minimum length : [Smallest subarray with sum greater than x](#), [Minimum Window Subsequence](#), [Minimum Window Substring](#), [Smallest containing 0, 1 and 2](#)
- Count Subarrays: [Subarrays with k odds](#), [Subarrays Less Than K](#), [Substrings with all vowels](#), [Number of Subarrays having sum less than K](#), [Count Subarrays with K Equal Value Pairs](#)
- Sliding Window with Hashing: [Longest substring with distinct](#), [Substring character replacement](#), [Longest Substring with K Uniques](#), [Substrings with k distinct](#), [Smallest with all characters](#), [Smallest distinct](#), [Count Anagrams](#), [Full Distinct Subarrays](#)

Prefix Sum

- Theory : [Prefix Sum](#)
- Prefix Sum: [Prefix Sum Array](#), [Prefix Sum on 2D](#), [Range Queries](#), [Equilibrium Index](#), [Mean of range](#), [Split array](#), [Largest Submatrix](#), [Max sum rectangle](#)
- Prefix / Suffix Transformation: [Product of Array Except Self](#), [Original Array](#), [Longest Span](#)
- Difference Array: [Diff Array](#), [2D Diff](#), [Max occurring in ranges](#), [Compute before Matrix](#), [Point Counts from Intervals](#)
- Prefix Sum with Hashing: [Subarray with 0 sum](#), [Largest with 0 sum](#), [Largest with Equal 0s and 1s](#), [Largest with sum divisible by k](#), [Longest with Sum K](#), [Count with Sum Divisible By K](#), [Count having Sum K](#), [Count with given XOR](#)

Backtracking

- Theory : [Backtracking](#)
- Medium : [Permutations](#), [Rat in a Maze](#), [Word Search](#)
- Hard : [N Queen Problem](#), [Sudoku Solver](#), [Word Break](#), [Knight's Tour](#), [Palindromic Partitions](#), [M-Coloring](#)

Linked-list

- Theory : [Linked-list](#), [Singly](#), [Doubly](#) & [Circular](#)
- Reversal Pattern: [Reverse Singly](#), [Reverse a Doubly](#), [Reverse in Groups \(K-group reversal\)](#), [Reorder List](#)
- Fast & Slow Pointer: [Middle](#), [Remove Nth from End](#), [Check Palindrome](#), [Detect Cycle](#), [Length of Loop](#), [Intersection of Y Shaped](#)
- Sorting & Merging Pattern: [Sort 0s, 1s, 2s](#), [Add Two Numbers](#), [Rotate](#), [Merge K Sorted](#)
- Design : [Browser History](#), [Twitter](#), [LRU Cache](#), [LFU Cache](#)
- Hard: [Flattening](#), [Clone with Random Pointer](#)

Stack

- Theory : [Stack](#)
- Operations : [Reverse](#), [Sort](#)
- Expression: [Check for balanced](#), [Evaluation of Postfix Expression](#), [Infix Evaluation](#), [Infix to Postfix](#), [Redundant Brackets](#), [Longest Valid](#), [Score of Balanced](#)
- Design: [Stack using Queues](#), [Two stacks in an array](#), [K Stacks in an Array](#), [Stack that supports getMin\(\)](#)
- Monotonic Stack: [Lowest by removing k digits](#), [Next Greater Element](#), [Stock Span Problem](#), [Buildings Facing Sun](#), [Previous Smaller Element](#), [Largest Rectangle in a Histogram](#), [Max of 1s](#), [Max of Mins of every window size](#), [Trapping Rain Water](#), [Subarray Range Sum](#), [Replace Adjacent Opposite](#), [Longest Subarray Length](#)
- String Simulation: [Decode String](#), [Check Redundant Brackets](#)

Queue

- Theory : [Queue](#)
- Easy : [Circular Queue](#), [Queue using Stacks](#), [FIFO Page Replacement](#), [Reverse a Queue](#), [Reverse First K](#)
- Medium : [Interleave Queue](#), [K Queues in an array](#), [First non-repeating in a stream](#), [Min Knight steps for target](#)

Deque

- Theory : [Deque](#)
- Easy : [Circular Array Implementation](#)
- Medium : [Lexicographically largest permutation](#), [Nth term of given recurrence relation](#), [Generate Bitonic Sequence](#)
- Hard : [Sliding Window Maximum](#), [Longest Subarray with Difference \$\leq X\$](#) , [Max Subarray Length with K Increments](#), [Min Deques to Sort Array](#)

Binary Tree

- Theory : [Binary Tree](#), [Inorder](#), [Preorder](#), [Postorder](#), [Level Order](#)
- Easy : [Size of Tree](#), [Depth of Tree](#), [Max Width](#), [Balance Check](#), [Identical Trees](#), [Mirror Tree](#), [Sum Tree](#), [Left View](#), [Right View](#), [Path Sum](#), [Max Path Sum](#), [Serialize and Deserialize](#),
- Iterative Traversals : [Inorder](#), [Preorder](#), [Postorder](#), [Morris Inorder](#)
- Medium : [Longest Sum](#), [Diameter](#), [Boundary Traversal](#), [Tree from Preorder and Inorder](#), [Leaf at Same Level](#), [Connect at Same Level](#), [Nodes without siblings](#), [Invert Tree](#), [Flatten Tree](#), [Max Diff Node & Ancestor](#)
- Horizontal Distance Based: [Vertical Traversal](#), [Bottom View](#), [Top View](#), [Diagonal Traversal](#)
- Hard : [Spiral Level Order](#), [LCA](#), [Distance K](#), [Distributed Candies](#), [Subtree](#)

Binary Search Tree

- Theory : [Binary Search Tree](#)
- Operations: [Search](#), [Insert](#), [Delete](#), [Ceil](#), [Floor](#), [Predecessor and Successor](#)
- Traversal: [Kth Smallest Element](#), [2 Sum in BST](#)
- Construction: [Sorted Array to BST](#), [Merge two BSTs](#)
- Property: [Validate BST](#), [LCA](#), [Recover BST](#), [Largest BST Subtree](#)

Heap

- Theory : [Heap](#)
- Easy : [Check for Heap](#), [Heap Sort](#), [kth Largest](#), [K Largest](#), [Height of a Heap](#)
- Medium : [Connect n ropes](#), [Nearly Sorted](#), [BST to Min Heap](#), [K Max sum combinations](#)
- Hard : [Median in a stream](#), [K'th largest in a stream](#), [Merge k sorted](#), [Range Covering K Sorted Lists](#), [The Skyline Problem](#), [Sort by Frequency](#)

Graph

- Theory : [Graph Representation](#)
- BFS/DFS Traversal: [BFS](#), [DFS](#), [Flood Fill](#), [Islands](#), [Connected Components in an Undirected Graph](#), [Cycle in an Undirected graph](#), [Cycle in a Directed](#), [Bipartite](#), [Rotten Oranges](#), [Remove Invalid Parentheses](#)
- Topological Sorting: [Topological Sorting](#), [Course Schedule I](#), [Course Schedule II](#), [Alien Dictionary](#)
- Shortest Path: [Dijkstra](#), [Floyd Warshall](#), [Bellman–Ford](#), [Minimizing Infection Time in a Directed Graph](#), [Shortest path with one curved edge in an undirected Graph](#), [Minimum Cost Path](#), [Shortest cycle in an undirected graph](#)
- Multi-Source BFS: [Distance of nearest 1](#), [Unlock the Lock](#), [Word Ladder](#)
- MST/DSU: [Prim](#), [Disjoint Set](#), [Kruskal](#), [Detect Cycle using DSU](#), [Total Spanning Trees](#), [Job Sequencing](#), [Number of connected components](#)

Greedy

- Theory : [Greedy](#)
- Easy : [Fractional Knapsack](#), [Assign Cookies](#), [Min and Max Costs to buy all](#), [Smallest Subset Greater Sum](#), [Min Notes with Given Sum](#)
- Medium : [Activity Selection](#), [Huffman Coding](#), [Jump Game](#), [Job Sequencing](#), [Non-Overlapping Intervals](#), [Gas Station](#)
- Hard : [Stable Marriage](#), [Minimize the Max Height Diff](#), [No two adjacent are same](#), [Minimize cash flow](#), [Min time to finish all jobs with given constraints](#), [Min Cost to cut into squares](#)

Dynamic Programming

- Theory : [Dynamic Programming](#)
- 1-D DP: [Climbing Stairs](#), [Count without consecutive 1s](#), [Weighted Climbing Stairs](#), [Frog Jump](#), [House Robber](#), [Max Segments](#), [Coin Change](#), [Possible Decodings](#), [Word Break](#), [Derangements](#)
- 2-D DP: [Unique Paths](#), [Minimum cost Path](#), [Longest Common Subsequence](#), [Edit Distance](#), [Distinct Subsequences](#), [Largest Square Submatrix with All 1s](#)
- SubSequence: [Partition Subset](#), [Subset Sum](#), [Subset Count With Sum](#), [Target Sum](#), [0/1 Knapsack](#), [Coin Change II](#), [Rod Cutting](#)
- Grid: [Unique Paths in a Grid](#), [Maximum path sum in matrix](#), [Longest Increasing Path](#)
- String: [Longest Common Substring](#), [Shortest Common Supersequence](#), [Interleaving String](#), [Wildcard Matching](#), [Longest Palindromic Substring](#)
- Stock: [Buy and Sell with Cooldown](#), [Buy and Sell – Max 2 Transactions Allowed](#), [Buy and Sell - k Transactions](#), [Buy and Sell with Transaction Fee](#)
- LIS: [Longest Increasing Subsequence](#), [Number of LISs](#), [Longest Bitonic](#), [Largest Divisible](#), [Longest String Chain](#), [Maximum envelopes](#)

- Partition: [Matrix Chain Multiplication](#), [Minimum Cost to Cut a Stick](#), [Palindrome Partitioning](#), [Boolean Parenthesization](#), [Partition Array for Maximum Sum](#), [Egg Dropping](#), [Optimal Strategy](#)

Number Theory

- Easy : [nCr](#), [Euler's Totient](#)
- Medium : [Sieve of Eratosthenes](#), [Pascal's Triangle](#), [Modular Exponentiation](#)
- Hard : [Segmented Sieve](#)

Trie

- Implementation: [Implement Trie](#), [Longest Word With All Prefixes](#), [Count of distinct substrings](#)
- Bit Manipulation Trie: [Duplicates in Binary Matrix](#), [Max XOR of Two](#), [Maximum XOR Queries](#), [Pairs with XOR less than K](#)
- Advanced Trie Application: [Search Suggestions System](#), [Palindrome Pairs](#)

String Matching

- Theory : [Substring](#)
- Standard Problems : [Rabin-Karp](#), [KMP](#), [Z algorithm](#), [Longest Prefix Suffix](#), [Manacher's Algorithm](#), [Subarray Search](#)
- Hard : [Longest Palindromic Substring](#), [Palindrome Substring Queries](#), [Min Repeats for Substring](#)

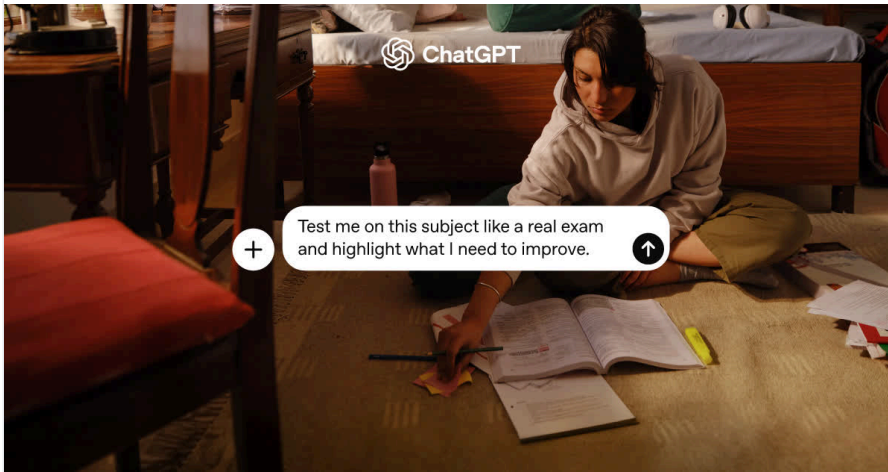
Range Query

- Basic Segment Tree Queries: [Segment Tree](#), [Min-Max Range Queries](#), [Range LCM Queries](#), [XOR of a given range](#)
- Segment Tree with Updates: [Lazy Propagation](#), [Flipping Sign](#)
- Segment Tree Applications: [Build a segment tree for N-ary rooted tree](#), [Maximum of all subarrays of size K using Segment Tree](#)
- Fenwick Tree: [Fenwick Tree](#), [Sum and Update Queries on 3D Matrix](#)

Topic Wise Complete Tutorials

Below are the recommended different topics to learn complete DSA.

ADVERTISING



Answers & Inspiration

[Learn More](#)

[View Questions](#) [Quizzes](#) [Must Do](#) [Advanced DSA](#) [System Design](#) [Aptitude](#) [Puzzles](#) [Interview C](#) [Search](#) [Sign In](#)

- Fundamentals : [Maths](#), [Complexity Analysis](#)
- Data Structures : [Array](#), [Hashing](#), [Strings](#), [Matrix](#), [Linked List](#), [Stack](#), [Queue](#), [Deque](#), [Tree](#), [Heap](#), [Graph](#)
- Algorithms : [Searching](#), [Sorting](#), [Two Pointer](#), [Window Sliding](#), [Prefix Sum](#), [Recursion](#), [Greedy Algorithms](#), [Dynamic Programming](#)
- Advanced Topics : [Trie](#), [Segment Tree](#), [Red-Black Tree](#) & [Binary Indexed Tree](#)
- Other Algorithms : [Bitwise Algorithms](#), [Backtracking](#), [Divide and Conquer](#), [Branch and Bound](#), [Geometric](#) & [Randomized](#)

Roadmap to learn DSA

[Visit Course](#)

[Comment](#)

R **Rishab...**

1.83k

Article Tags:

[DSA](#)

[Tutorials](#)

[DSA Tutorials](#)

 Corporate & Communications Address:

A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

 Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company

[About Us](#)

[Legal](#)

[Privacy](#)

[Policy](#)

[Careers](#)

[Contact Us](#)

[Corporate](#)

[Solution](#)

[Campus](#)

[Training](#)

Explore

[POTD](#)

[Practice](#)

[Problems](#)

[Connect](#)

[Blogs](#)

[Upskill](#)

[Courses](#)

Tutorials

[Programming](#)

[Languages](#)

[DSA](#)

[Web](#)

[Technology](#)

[AI, ML &](#)

[Data Science](#)

[DevOps](#)

[CS Core](#)

[Subjects](#)

[GATE](#)

[School](#)

[Subjects](#)

[Software and](#)

[Tools](#)

Courses

[ML and Data](#)

[Science](#)

[DSA and](#)

[Placements](#)

[Web](#)

[Development](#)

[Data Science](#)

[Programming](#)

[Languages](#)

[DevOps &](#)

[Cloud](#)

[GATE](#)

[MongoDB](#)

[Certifications](#)

Preparation Corner

[Interview](#)

[Corner](#)

[Aptitude](#)

[Puzzles](#)

[GfG 160](#)

[System Design](#)